

Project Handover Document

Project Name

A GraphRAG-Powered Personalized Learning Platform for Medical Students

Project Contributors

- Michael Eko Hartono Gunawan
- Ghazali Ahlam Jazali
- Darrell Cornelius Rivaldo

Stakeholders

- dr. Florence Pribadi, M.Sit.
- dr. Maria Jessica Rachman, M.Si.

Project Period

Start Date : 15-07-2026
End Date : 21-08-2026

Deliverable Checklist

Item	Location
Raw Data	Link to artifact location
Preprocessed Data	Link to artifact location
Model Experiment Notebooks	Link to artifact location
Apps Source Code	Link to artifact location
License	Link to artifact location
Access Ownership	Link to artifact location

Executive Summary

Project Overview

Problem Statement

First-year medical students at Ciputra University differ in their strengths, weaknesses, and learning pace. Although a standardized curriculum ensures that every student receives the same foundational learning materials, lectures are typically delivered at a uniform pace and level of difficulty. This limits the ability of classroom instruction to accommodate individual learning differences, leaving some students with unresolved knowledge gaps. Therefore, additional personalized learning activities are needed to support each student's specific learning needs.

Solution

To address the varying strengths, weaknesses, and learning pace of first-year medical students, we propose an adaptive learning system that continuously estimates each student's mastery of medical concepts and recommends personalized practice questions and flashcard reviews. By tailoring learning content to individual needs, the system helps students focus on concepts they have not yet mastered, improving learning efficiency and supporting more effective knowledge acquisition.

Result

As the solution to personalized medical learning, we developed **MediQuiz**, an Intelligent Tutoring System (ITS) and adaptive study application built natively for iPadOS. The application serves as an interactive study companion for first-year medical students, translating dense medical textbook curricula into personalized practice quizzes, spaced-repetition flashcards, and real-time knowledge tracking.

Application Overview & Core Features

MediQuiz provides an intuitive and distraction-free learning environment designed to support self-directed study through the following features:

- **Adaptive Practice Quizzes:** Students can generate tailored practice quizzes from specific medical subjects (e.g., Physiology, Biochemistry, Histology, Anatomy) or construct custom multi-topic study sessions. Questions feature multiple difficulty levels ranging from fundamental recall to clinical case vignettes.
- **Confidence-Based Immediate Feedback:** While answering questions, students indicate their self-reported confidence level (*Confident*, *Unsure*, or *Guessing*). Upon selection, the application provides instant verification along with comprehensive clinical explanations to reinforce underlying concepts and correct misconceptions.
- **Spaced-Repetition Flashcard Decks:** Students can review core medical facts through interactive flashcards with flip animations. By rating their recall difficulty (*Again*, *Hard*, *Good*, *Easy*), the platform dynamically calculates optimal future review dates to maximize long-term retention.

- **Curriculum Exploration & Mastery Tracking:** An interactive subject directory provides students with visual progress bars indicating their mastery level across each topic and sub-topic, making it effortless to identify weak areas and track learning progression over time.
- **Smart Dashboard & Study Nudges:** The main dashboard delivers actionable study recommendations (“nudges”) that prioritize urgent review items and topics requiring reinforcement. Daily streak counters and gamified energy rewards encourage consistent daily practice.
- **Review History & Bookmarking:** Students can bookmark challenging questions for focused revision and review past quiz attempts and historical performance summaries.

Model Outcomes & Validation Results

The initial evaluation and operational benchmarks of the AI pipeline and adaptive models showed the following results:

- **Knowledge Tracing Baseline:** On initial interaction trajectories, the Bayesian Knowledge Tracing model achieved a **ROC-AUC of 0.6819** for next-step correctness prediction, offering a functional baseline for estimating relative topic mastery and surfacing potential knowledge gaps.
- **Graph-Grounded Question Synthesis:** Leveraging the biomedical knowledge graph pipeline (**knowledge-graph-pipeline**), questions are synthesized directly from asserted semantic triples across a 27-relation medical ontology. Each item anchors to a verified source sentence as an auditable rationale, using indefinite stem framing to prevent open-world factual inaccuracies.
- **Multi-Tier Distractor Safety & Semantic Guardrails:** To ensure distractors are plausible yet unambiguously incorrect, the pipeline enforces a multi-tier defense: filtering against global relation blocklists (asserted, inferred, and flagged edges), excluding cross-related concepts, and applying deterministic relation guards that prevent clinically inverted claims (such as distinguishing nutrient deficiencies from the nutrients themselves).

AI/ML Solution Technical Overview

Data Collection & Preprocessing

Data Sources

- **Medical Textbooks (data/textbook/):**
 - **Source & Format:** AccessMedicine by McGraw-Hill (PDF, 3 documents, ~700–1000 pages each)
 - **Titles:**
 - * *Ganong’s Review of Medical Physiology (27th ed.)*
 - * *Harper’s Illustrated Biochemistry*
 - * *Junqueira’s Basic Histology: Text and Atlas*
 - **Content Structure:**
 - * Chapter titles, headings, and sub-headings
 - * Paragraph body text and nested list structures
 - * Tables with captions and footnotes
 - * Diagrams and flowcharts
 - * Inline mathematical equations
- **Question Sets (data/mcq/):**
 - Pre-made MCQ evaluation packets (*Batch 1*, *Batch 2*) for question validation and benchmarking.

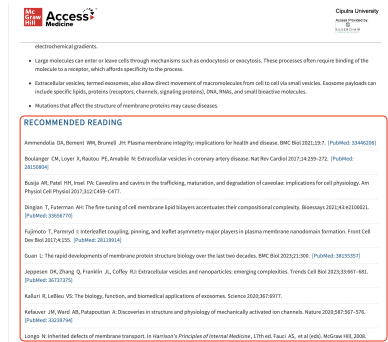
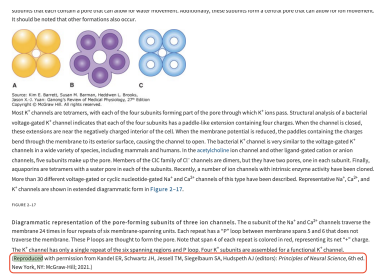
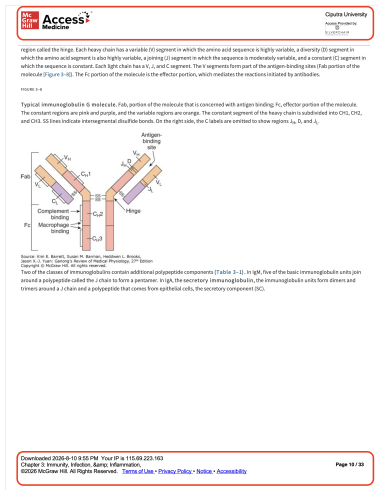
Data Cleaning

Raw PDFs are first extracted by **MinerU** (in an isolated Python environment), which produces a structured `*_middle.json` block file and associated `*_content_list.json` per chapter. The `post_process_book` package then cleans this MinerU output to produce the final text corpus.

- Removing irrelevant content blocks: image blocks, running headers/footers, page numbers, source attribution lines, “reproduced/adapted with permission” phrases, publisher citation parentheticals, and entire *Recommended Reading / References* sections.
- Preserving all paragraph text, list structures, table bodies, inline figure references (e.g. *Figure 1-1*), and mathematical equations in their original form.
- Outputs per chapter: `<basename>.cleaned.md` (full cleaned markdown) and `<basename>.page_map.json` (character-level and line-level page offsets used for downstream retrieval grounding).

Data Preprocessing

After cleaning, the `.cleaned.md` files pass through two sequential steps before knowledge-graph construction.



- **Document rewriting:** `rewrite_medical_md.py` (document-rewriter repository) reads each `.cleaned.md` chapter and re-emits it as a structurally tagged markdown file. Body paragraphs are carried over verbatim into `<con>` elements; PDF-extraction artefacts (non-breaking spaces, soft hyphens, mid-sentence page-break splits) are repaired. Inference is limited to one **HuatuoGPT-o1-72B** call per chapter for the `<topic>` element, which has no source counterpart.
- **Stage 1 parse:** The tagged markdown is consumed by Stage 1 of the `medkg` pipeline (`stage1_parse.py`), which produces a tag-free normalised text file and a structured IR JSON carrying offset-anchored chunks, figures and passages. Chunk kinds are `prose`, `definition`, `clinical`, `summary` and `caption`; only the first four reach Stage 2 extraction.

Feature Engineering

- **Context enrichment:** The most specific section heading from each chunk's `section_path` is prepended to its text before any LLM call, preserving the governing medical concept even for short fragment chunks.
- **Medical triple extraction:** Stage 2 of the `medkg` pipeline makes one **HuatuoGPT-o1-72B** LLM call per chunk to extract structured `{subject, relation, object}` triples. Relations are constrained to a closed **27-relation taxonomy** defined in `medkg/ontology_medschool.json`: `treated_with`, `prevented_by`, `caused_by`, `finding_site`, `associated_with`, `present_in_patient`, `part_of`, `located_in`, `composed_of`, `connects_to`, `secretes`, `synthesizes`, `stores`, `transports`, `converts_to`, `catalyzes`, `regulates`, `stimulates`, `inhibits`, `requires`, `has_function`, `risk_factor_for`, `complication_of`, `diagnosed_by`, `adverse_effect_of`, `contraindicated_in`, `has_mechanism`. 14 deterministic relation guards then audit every extracted relation before assertion.
- **Distractor engineering:** A two-tier strategy selects wrong answer options. Tier 1 performs corpus-level retrieval — for the same `(topic, relation)` pair, other objects seen elsewhere in the corpus serve as plausible distractors. Tier 2 falls back to the LLM when Tier 1 yields fewer than 3 candidates. A `Pydantic DistractorSet` validator rejects any distractor that duplicates the correct answer.
- **Domain Knowledge Graph:** `medkg/` implements an 8-stage pipeline that builds an RDF knowledge graph from the textbook markdown — `scispaCy NER`, `SapBERT/FAISS`

entity linking to UMLS/SNOMED CT, deterministic post-coordination modifier minting (INSTANCE nodes), and RDFS/OWL-RL reasoning. This enriches the semantic structure of extracted medical entities beyond what the triple-extraction LLM alone can capture.

Exploratory Data Analysis

- **Question bank distribution:** The generated question bank (`question_bank.json`) contains **188 validated MCQs** spanning medical textbook sections, distributed across three difficulty tiers: Tier 1 (Recall), Tier 2 (Application), and Tier 3 (Clinical Vignette Reasoning).
- **Self-critique correction rate:** Approximately **62% (116 of 188)** questions triggered the automated self-critique correction pass (`valid_as_generated: false`), demonstrating that the *Critique* and *Corrector* feedback loop is essential for output quality — single-pass generation alone was insufficient.
- **MinerU quality diagnostics:**
`src/evaluation/mineru_quality.py` provides reference-free quality metrics for raw MinerU output, including garbage-character ratio, empty-block ratio, repeated-character runs, and per-block confidence scores. These were used to spot-check extraction quality across the five textbook PDFs.
- **Knowledge graph structure:** Student interaction data is stored in Neo4j as (Student)-[:ANSWERED]->(Question) edges, with a topic hierarchy modelled as (Topic)-[:SUB_TOPIC_OF]->(Topic) to support spaced-repetition scheduling and Bayesian Knowledge Tracing queries.

Model Experiment

All models are deployed in an **inference-only** setup — no fine-tuning or backpropagation during live serving. The list below documents each model’s role, architectural hyperparameters, operational runtime configuration, fallback defaults, and validation mechanisms across the pipeline:

- **HuatuoGPT-o1-72B (4-bit)** (*Medical Reasoning LLM*)
 - **Role:** Document rewriting and semantic tagging (`rewrite_medical_md.py`); Stage 2 relation and modifier extraction (`medkg/stage2_extract.py`); Stage 3 ambiguous concept reranking (`medkg/stage3_link.py`).
 - **Configuration & Hyperparameters:**
 - * *Checkpoint & Backend:* `mlx-community/HuatuoGPT-o1-72B-4bit` served via an OpenAI-compatible vLLM/MLX endpoint (`DEFAULT_BASE_URL = "http://jupyter-02.aml1.id.iosda.org:8888"`, default fallback `http://localhost:8888`).
 - * *Rewriter Sampling:* Section-specific temperatures: $T_{\text{topic}} = 0.30$, $T_{\text{caption}} = 0.15$ (strict factual alignment), $T_{\text{objectives}} = 0.35$, $T_{\text{summary}} = 0.35$, fallback $T = 0.40$; $\text{top-}p = 0.90$.
 - * *Rewriter Token Limits:* `max_tokens = 1024` for `<topic>` and `<cap>`; `max_tokens = 2048` for `<obj>`, `<sum>`, and general completions.

- * *Stage 2 Extraction*: Greedy decoding ($T = 0.0$, `max_tokens = 4096`) with vLLM guided JSON decoding (`extra_body={"guided_json": schema}`).
 - * *Stage 3 Disambiguation*: Greedy decoding ($T = 0.0$, `max_tokens = 24`) with guided choice (`extra_body={"guided_choice": choices}`), cached per (`mention, choices`) tuple.
 - * *Execution & Resilience*: 900s timeout (rewriter) / 300s timeout (Stage 2); 4 retries with exponential backoff; 4 concurrent worker threads; SHA-256 payload disk caching; client-side chain-of-thought stripping (`split_reasoning` discarding `## Thinking` and `<think>` traces); circuit breaker halting after 5 consecutive failures (`BackendUnavailable`); `-offline` rule-based fallback.
 - **Validation**: Tag-completeness checks; exact-match chunk text grounding; Pydantic schema validation (`Triple, Modifier`); closed 27-relation taxonomy guards.
- **SapBERT** (*Biomedical Sentence Transformer*)
 - **Role**: Stage 3 entity linking to UMLS/SNOMED CT concepts (`medkg/stage3_link.py`).
 - **Configuration & Hyperparameters**:
 - * *Checkpoint & Pooling*: `cambridgeltl/SapBERT-from-PubMedBERT-fulltext` ($d = 768$); `[CLS]` token representation from `last_hidden_state`; unit L_2 -normalization.
 - * *Index & Search*: FAISS `IndexFlatIP` (Exact Inner Product on normalized vectors $\equiv$ Cosine Similarity); candidate retrieval depth $k = 20$; sequence length capped at 32 tokens; batch size 256.
 - * *Thresholds*: Cosine similarity floor ≥ 0.70 (`LINK_CONFIDENCE_FLOOR = 0.70`); LLM reranker invoked only when top-2 candidate margin ($\text{score}_1 - \text{score}_2$) < 0.05 (`RERANK_MARGIN = 0.05`).
 - * *Concurrency Safety*: Native thread pinning (`MEDKG_NATIVE_THREADS = 1`) across OpenMP, PyTorch, and FAISS runtimes to prevent macOS Darwin/BLIS segmentation faults.
 - * *Semantic Typing*: UMLS `MRSTY.RRF` crosswalk (`semantic_types.json`) applying `SEMANTIC_TYPE_PRIORITY` to refine coarse NER labels into canonical ontology span labels.
 - **Validation**: Link-confidence distribution; CUI collision detection; semantic type compatibility checks.
 - **Qwen3.5-27B / Qwen3.5-9B-OptiQ** (*LLM*)
 - **Role**: MCQ question-stem phrasing, distractor evaluation, and automated self-critique loop (`medkg/mcq.py`).
 - **Configuration & Hyperparameters**:
 - * *Checkpoint & Backend*: `Qwen/Qwen3.5-27B` or `mlx-community/Qwen3.5-9B-OptiQ-4bit` via OpenAI-compatible endpoint (`http://localhost:8888/v1`); $T = 0.0$, `max_tokens = 4096`, guided JSON schema decoding.
 - * *Batching & Limits*: Batch size of 10 items per phrasing pass (`phrase_items`); maximum stem length capped at 300 characters; 5 options per question (1 key + 4 distractors).
 - * *Distractor Safety Guardrails*: 3-tier open-world exclusion (asserted/inferred/flagged triple blocklist; alternate head-edge prohibition;

- indefinite stem framing); negative-stem regex guard (`_NEGATIVE_RE`) rejecting negative phrasing to prevent open-world fallacies.
 - * *Self-Critique Cycle*: Automated critique and corrector loop evaluating distractor plausibility and answer leakage.
 - **Validation**: Pydantic schema validation (`DistractorSet`, `Critique`); negative-stem guard; stem character ceiling.
- **Qwen2.5-VL-7B-Instruct & 3B-Instruct** (*Vision-Language Model*)
 - **Role**: Textbook diagram/figure visual captioning (`src/captioning/qwen_vl.py`), full-page OCR transcription (Thread B), and exam PDF question extraction.
 - **Configuration & Hyperparameters**:
 - * *Precision & Placement*: `torch.bfloat16`; two-stage load (CPU → MPS/CUDA) bypassing Apple Silicon MPS fp16 kernel crashes; lazy instantiation with explicit garbage collection and cache clearance (`torch.mps.empty_cache()`).
 - * *Visual Captioning*: `max_new_tokens = 512`, deterministic greedy decoding (`do_sample = False`), grounded with MinerU OCR labels.
 - * *Page Transcription*: `max_new_tokens = 2048`, `max_pixels = 1,280,000` (1.28M px) bounding vision prefill token overhead on MPS.
 - * *Image Relevance*: `max_new_tokens = 5`, `max_pixels = 480,000`, single-token classification (`SEMANTIC` vs. `DECORATIVE`).
 - **Validation**: Visual grounding against MinerU OCR text labels; single-word relevance classification output validation.
- **scispaCy & NegEx** (*Biomedical NER & Negation Detection*)
 - **Role**: Stage 2 entity recognition and negation detection (`medkg/stage2_extract.py`).
 - **Configuration**: Multi-model pipeline (`en_ner_bc5cdr_md` for diseases/chemicals, `en_ner_bionlp13cg_md` for anatomy/cells/proteins); `AbbreviationDetector` and `Negex` over all entity types; longest-span-first entity merging.
 - **Validation**: Cross-model span deduplication; ontology label mapping per `NER_LABEL_MAPS`.
- **all-MiniLM-L6-v2** (*Sentence Transformer*)
 - **Role**: Semantic boundary chunking of unified text streams (`src/ingestion/semantic_chunk.py`).
 - **Configuration**: Cosine distance threshold calculated at the 95.0 percentile over each document; hard fallback split at 6000 characters.
 - **Validation**: Distributional percentile adaptation per textbook writing style.
- **Bayesian Knowledge Tracing & Spaced Repetition Algorithms** (*Probabilistic & Cognitive Models*)
 - **Role**: Real-time student mastery tracking and personalized quiz/flashcard review scheduling (`src/quiz/`, `src/flashcards/`, `BKTStore.swift`).
 - **Configuration & Hyperparameters**:
 - * *BKT Engine*: 2-state Hidden Markov Model running on-device in Swift with bootstrap priors: $p_{\text{init}} = 0.30$, $p_{\text{transit}} = 0.10$, confidence-conditional slip p_{slip} (0.05 confident / 0.10 unsure / 0.30 guessing), and guess p_{guess} (0.10 confident / 0.25 unsure / 0.50 guessing); synced via PUT `/quiz/mastery` to Neo4j.

- * *Spaced Repetition (Quiz)*: Confidence-weighted interval progression on (:Student)-[:REVIEWING]->(:Question): correct + confident doubles interval ($1 \rightarrow 2 \rightarrow 4 \rightarrow \dots \leq 60$ days); wrong or unsure resets streak to 0 (interval 1 day).
- * *Spaced Repetition (Flashcard)*: 4-point rating schedule on (:Student)-[:HAS_FLASHCARD]->(:Flashcard): *Again* (0.0 / 1 day), *Hard* ($1.2\times$), *Good* ($2.0\times$), *Easy* ($2.5\times$), capped at 60 days.
- **Validation**: Empirical ROC-AUC next-step prediction evaluation (0.6819) and review interval convergence monitoring.

Conclusion

- **Final system architecture**: MinerU PDF extraction $\rightarrow$ `post_process_book` cleaning (`.cleaned.md` + `.page_map.json`) $\rightarrow$ `document-rewriter` (HuatuoGPT-o1-72B tagging) $\rightarrow$ `medkg` Stages 1–8 (parse $\rightarrow$ NER $\rightarrow$ entity linking $\rightarrow$ relation extraction $\rightarrow$ post-coordination $\rightarrow$ RDF assertion $\rightarrow$ OWL-RL reasoning $\rightarrow$ N-Quads) $\rightarrow$ Qwen3.5-27B MCQ generation pipeline. A decoupled FastAPI serving layer reads from PostgreSQL (`mcq_questions` table) and logs student interaction graphs in Neo4j, with **zero GPU/PyTorch runtime dependencies** in the live API.
- **Why it performs best**:
 - *Decoupled architecture*: The live API is fully CPU-bound; all heavy inference runs offline during ingestion and generation.
 - *High quality control*: Every served question passes strict Pydantic schema validation and the self-critique correction loop before being committed to the database.
 - *Graph-based student tracking*: Neo4j enables real-time spaced repetition and Bayesian Knowledge Tracing (BKT) based on per-topic mastery edges.
- **Comparison with alternative approaches**:
 - *Single-pass LLM question generation*: Replaced by the 6-step triple extraction $\rightarrow$ distractor generation $\rightarrow$ self-critique pipeline to eliminate answer leakage and reduce hallucination in generated stems.
- **Final performance metrics**:
 - *BKT knowledge tracing validation*: Next-step correctness prediction evaluated on student interaction trajectories achieved an empirical **ROC-AUC of 0.6819** and a log-loss of **0.7105**, confirming its effectiveness in tracking evolving concept mastery and surfacing student knowledge gaps.
 - *Question bank coverage*: Total of **188 validated MCQs** generated and structured across three cognitive difficulty tiers.
 - *Automated critique refinement*: **62% (116 of 188 questions)** successfully refined via the self-critique verification loop.
 - *Ingestion throughput*: `post_process_book` cleaner executes deterministically in sub-second time per PDF chapter.

AI/ML System Technical Overview

System Architecture

Architectural Philosophy: The “Two Halves” Principle

The system is strictly partitioned into two decoupled halves that interact exclusively through persisted storage contracts:

- **The Offline Content & Knowledge Pipeline (Heavy AI/ML):**
 - Ingests raw textbook PDFs and slides.
 - Executes multimodal OCR, vision-language diagram parsing, rule-based AST transformations, scispaCy/SapBERT medical entity linking, and multi-step LLM question generation with self-critique.
 - Requires CUDA/MPS GPU hardware, PyTorch, Transformers, FAISS, and isolated Python runtimes.
 - Produces clean, validated relational tables in PostgreSQL (`mcq_questions`) and semantic graphs in Neo4j / RDF (`graph.nq`).
- **The Live Serving Layer (FastAPI + iPadOS):**
 - High-throughput, CPU-bound REST API built on FastAPI and Uvicorn.
 - **Zero PyTorch or Transformers runtime dependencies** — the serving layer never loads model weights, eliminating GPU hosting costs, cold-start latency, and out-of-memory crashes on web workers.
 - Backed by PostgreSQL for immutable question content and Neo4j for event-sourced student interaction history, spaced repetition scheduling, and topic mastery.
 - Real-time personalization executes directly on the student’s iPad using an on-device Bayesian Knowledge Tracing (BKT) engine.

High-Level System Diagram

Service Layer & Component Decomposition

Backend Architecture (`intelligent-tutoring-system-for-medical-student`):

- **`app.main`:** Application entry point; installs middleware stack, CORS, SlowAPI rate limiting, and exception handlers.
- **`app.auth_middleware`:** Inspects every request against a strict regex path allowlist; resolves opaque Bearer tokens to `student_id` via Neo4j.
- **`app.errors`:** Formats all HTTP, validation, and driver exceptions into a unified contract: `{"error": {"code": "...", "message": "...", "details": [...]}}`.
- **`src.student_kg`:** Graph driver and node primitives for student records, streak calculations, gamified energy balances, and auth sessions.
- **`src.quiz`:** Database bank queries, session tracking, answer recording, spaced-repetition schedules, and Baum-Welch EM parameter fitting.

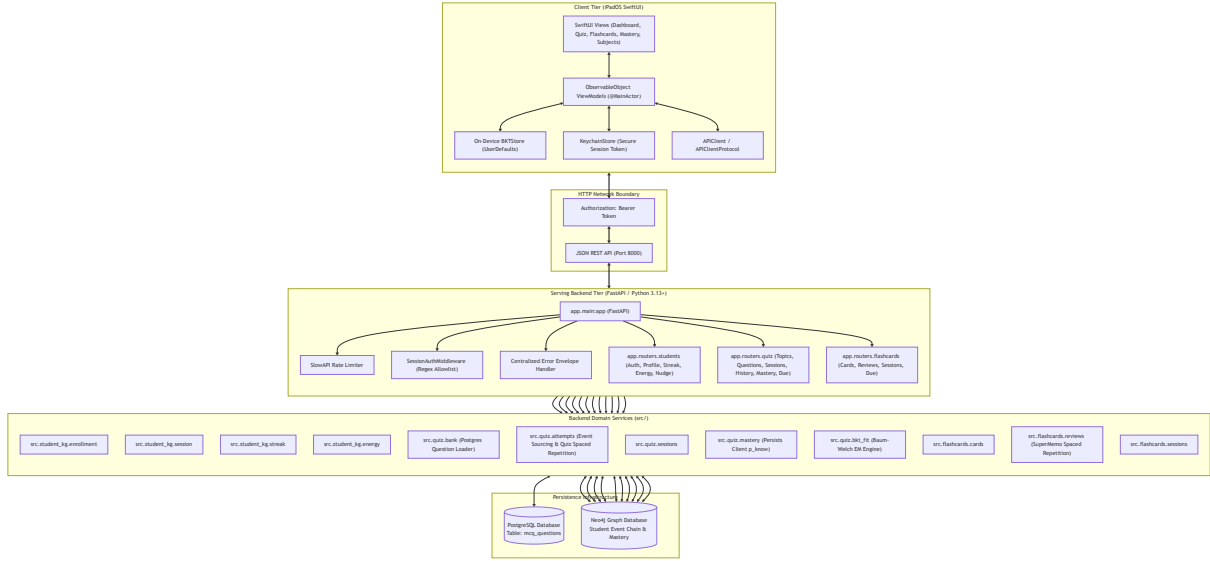


Figure 1: Full component decomposition: Client Tier, HTTP Network Boundary, Backend Tier, Domain Service Tier, and Persistence Infrastructure.

- **src.flashcards:** Flashcard generation from questions, 4-tier rating logging, and due-review queues.
- **src.post_process:** 8-stage deterministic cleanup pipeline for MinerU PDF parser outputs.
- **src.captioning:** Vision-language captioning using Qwen2.5-VL-7B.
- **src.domain_kg:** 8-stage biomedical knowledge graph extraction and reasoning pipeline (medkg).

Frontend Architecture (personalized-learning-ios):

- **Pattern:** MVVM (Model-View-ViewModel) + Protocol-driven Networking.
- **App Layer:** Application lifecycle, root state routing (`ContentView`), and navigation container (`RootView`).
- **Core Layer:**
 - *Networking:* `APIClient` implementing `APIClientProtocol`, `APIClientConfig`, `APIError`, and custom ISO8601 date decoding.
 - *Persistence:* `KeychainStore` (secure credential store), `SessionManager` (session token façade), and `BKTStore` (on-device Bayesian Knowledge Tracing engine).
 - *UI:* Global typography, palette tokens (`Theme`), `SidebarView`, and `TopBarView`.
- **Features Layer:** Encapsulated feature domains — *Auth*, *Dashboard*, *Quiz*, *Flashcard*, *Subjects*, *History*, *Bookmark*, *Settings* — each containing dedicated Views, ViewModels, API extensions, and Models.

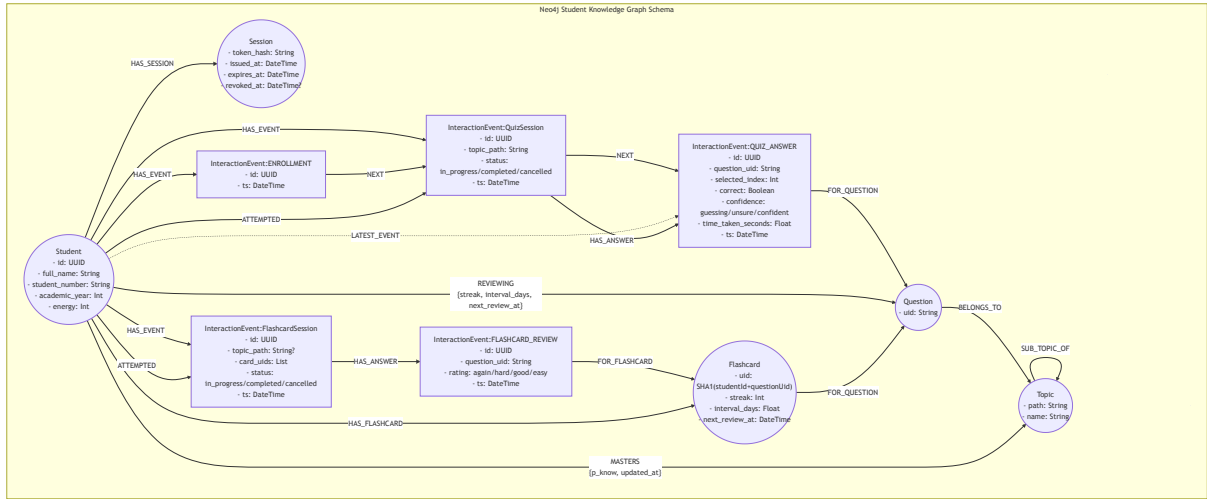


Figure 2: Neo4j Student Knowledge Graph Schema: nodes, relationships, and property contracts.

Storage & Graph Topology

The persistence model leverages PostgreSQL for relational data and Neo4j for event-sourced graph data.

Key graph design rules:

1. **Event Sourcing:** Every student interaction creates an immutable **InteractionEvent** node linked in chronological order via `:NEXT` edges and grouped under `:QuizSession` / `:FlashcardSession` via `:HAS_ANSWER`.
2. **Mutable Schedules as Edges:** Spaced repetition schedules are stored directly on the **(Student)-[:REVIEWING]->(Question)** edge (quiz) and the **Flashcard** node (flashcards), updating deterministically upon each submission.
3. **Mastery Persistence:** The `:MASTERS` edge represents the student's on-device BKT estimate (p_{know}), synchronised asynchronously via `PUT /quiz/mastery`.
4. **Relational Question Isolation:** The **Question** node in Neo4j contains **only** uid. Stem text, options, explanations, and diagrams reside in PostgreSQL to protect graph traversal performance.

Security, Authentication & Session Model

Why Opaque Bearer Tokens (Not JWT)?

JWTs cannot be instantly revoked without a distributed blacklist. Storing a SHA-256 hashed token in Neo4j allows instantaneous server-side revocation on logout (`revoked_at = datetime()`) while keeping raw tokens out of the database.

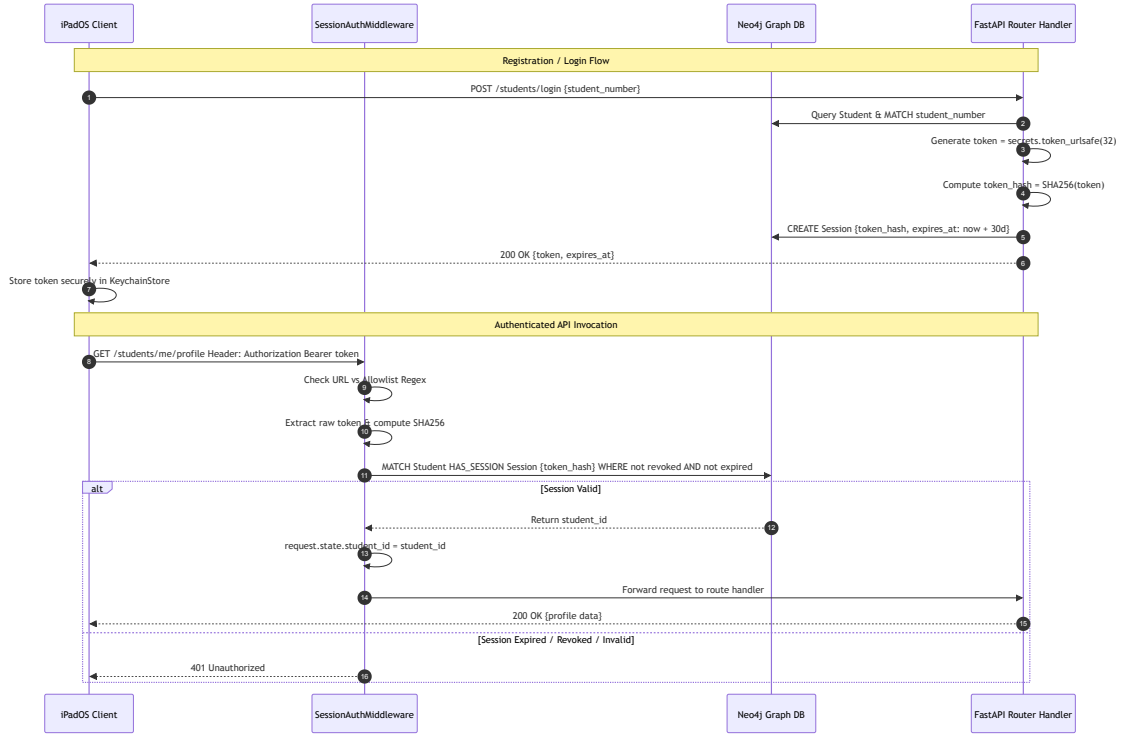


Figure 3: Opaque Bearer Token authentication sequence: registration/login flow and authenticated API invocation with `SessionAuthMiddleware`.

End-to-End Application Flows

Flow 1: Complete System End-to-End Dataflow

The application architecture coordinates three decoupled subsystems spanning offline content preparation, batch parameter estimation, and real-time interactive learning:

1. **Offline Content Ingestion & AI Generation:** Medical textbook PDFs are parsed via the MinerU CLI to extract bounding-box layouts and visual figures. Cleaned text (processed via an 8-stage rule-based AST engine) and multimodal figure captions (generated by Qwen2.5-VL-7B-Instruct) are unified into structured Markdown. Semantic chunking (all-MiniLM-L6-v2) feeds a 6-step LLM question generation pipeline (Qwen3.5-27B / HuatuoGPT-o1-72B), storing finalized questions in PostgreSQL (mcq_questions).
2. **Bayesian Knowledge Tracing (BKT) Parameter Training:** Student interaction events (:QUIZ_ANSWER) recorded in Neo4j are pooled for global parameter fitting. The server executes the Baum-Welch Expectation-Maximisation (EM) algorithm (src/quiz/bkt_fit.py) to update population-wide transition, slip, and guess probabilities, exposing them through GET /quiz/mastery/params.
3. **Live Student Learning Interaction:** The iPadOS client authenticates with the FastAPI backend, caching session tokens in Keychain. Students engage with interactive Quiz and Flashcard modules with sub-15 ms instant feedback, on-device Bayesian mastery updates, and asynchronous graph persistence to Neo4j.

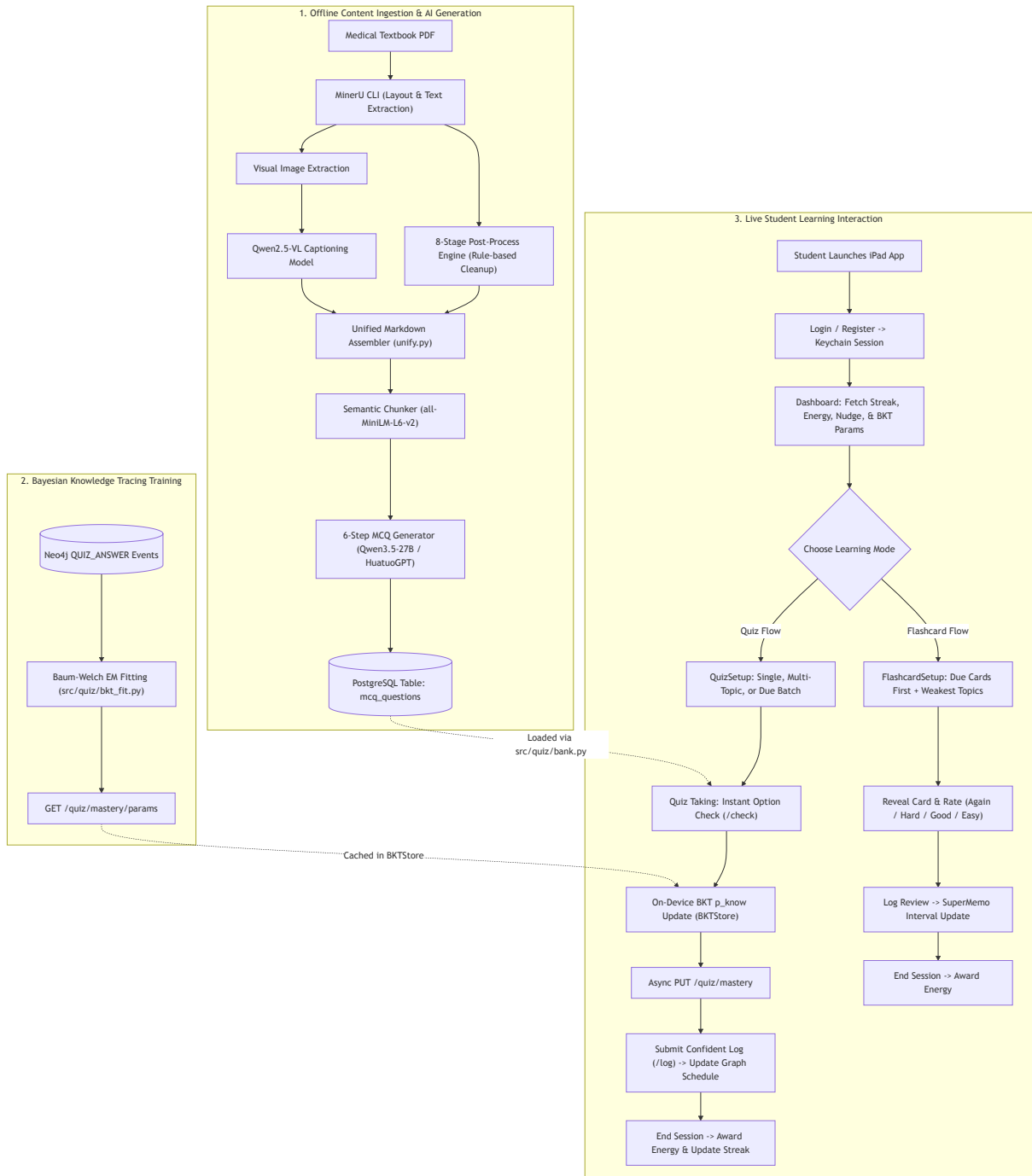


Figure 4: Complete system dataflow across the three subsystems: Offline Content Ingestion, BKT Parameter Training, and Live Student Learning Interaction.

Flow 2: Student Authentication & Session Lifecycle

The iPadOS client manages student identity through an opaque Bearer token architecture:

1. **Registration & Login:** The student submits credentials via `POST /students/login` or `POST /students/register`. The backend matches or creates the `:Student` node in Neo4j, mints a 32-byte cryptographic token (`secrets.token_urlsafe(32)`), computes its SHA-256 hash, and stores a `:Session` node with a 30-day expiration timestamp (`expires_at`).
2. **Secure Client Storage:** The raw token is returned to iPadOS and stored in `KeychainStore` using the Apple Security framework, while `SessionManager` publishes the active authentication state.
3. **Middleware Verification:** Subsequent HTTP requests provide the token via the `Authorization: Bearer <token>` header. `SessionAuthMiddleware` intercepts requests, computes the SHA-256 hash, and performs a single Cypher lookup against Neo4j to confirm the session is unrevoked and unexpired, injecting `student_id` into `request.state`.

Flow 3: Quiz Execution, Grading & Mastery Sync

The quiz lifecycle operates across four sequential phases designed to deliver instant feedback while guaranteeing asynchronous persistence:

1. **Option Selection & Instant Grading:** The student selects an answer option and confidence level (*Confident / Unsure / Guessing*). `POST /quiz/questions/{uid}/check` is a public, stateless endpoint that evaluates the option against an in-memory sanitized question bank—returning `correct`, `correct_index`, and clinical `explanation` in sub-15 ms.
2. **On-Device BKT Update:** `BKTStore.record()` computes the Bayesian posterior mastery p_{know} using confidence-stratified slip and guess probabilities. The updated estimate is written to local `UserDefaults` and synchronised to Neo4j via an asynchronous background `PUT /quiz/mastery` call.
3. **Stateful Log Commit:** When navigating to the next question, the client issues an authenticated `POST /quiz/questions/{uid}/log` request. The backend commits an immutable `:QUIZ_ANSWER` event linked to the active `:QuizSession` and updates the spaced-repetition schedule on the `(Student)-[:REVIEWING]->(Question)` edge.
4. **Quiz Completion & Gamification:** `POST /quiz/sessions/{id}/end` finalises the session, aggregates score metrics, updates the student’s daily activity streak, and awards +5 Energy currency in Neo4j.

Flow 4: Flashcard Spaced Repetition Lifecycle

Flashcard study sessions employ a modified SuperMemo spaced repetition algorithm tailored to student knowledge state:

1. **Deck Assembly & Weakness Prioritisation:** When initiating a session via `POST /flashcards/sessions`, the backend queries due cards where `next_review_at ≤ now`. Any remaining slots in the batch are dynamically filled with cards corresponding to the student’s lowest BKT mastery topics (p_{know}).

2. **Card Front & Answer Reveal:** The client presents the question stem and prompt. Tapping “Reveal Answer” dispatches `POST /flashcards/cards/{uid}/reveal` to retrieve the detailed answer back and explanation.
3. **Recall Rating & Interval Update:** The student evaluates recall quality on a 4-point scale (*Again, Hard, Good, Easy*). `POST /flashcards/cards/{uid}/log` records a `:FLASHCARD_REVIEW` node, recalculates interval days and repetition streaks on the `:Flashcard` node, awards +1 Energy, and returns the next review date.

Flow 5: Adaptive Knowledge Tracing & Personalization Loop

The personalization engine implements a closed-loop hybrid architecture combining server-side population fitting with client-side real-time state tracking:

1. **Server-Side Population Parameter Fitting (Periodic Batch):** Interaction sequences are extracted from pooled `:QUIZ_ANSWER` nodes in Neo4j. The Baum-Welch EM script (`src/quiz/bkt_fit.py`) executes Forward-Backward rescaling (E-step) and expectation maximisation (M-step) to derive global parameters: p_{init} , p_{transit} , and confidence-conditioned $p_{\text{slip}}(c)$ and $p_{\text{guess}}(c)$. Fitted parameters are cached and exposed via `GET /quiz/mastery/params`.
2. **Client-Side Real-Time Knowledge Tracing (iPadOS):** On application launch, `BKTStore` refreshes and caches population parameters. When the student answers a question with confidence c , `BKTStore` computes the closed-form Bayesian posterior:

$$p(L_t \mid y_t, c) = \frac{p(L_{t-1}) \cdot P(y_t \mid L_t, c)}{P(y_t \mid c)} \quad (1)$$

and applies the transition prior $p(L_{t+1}) = p(L_t \mid y_t, c) + (1 - p(L_t \mid y_t, c))p_{\text{transit}}$. The resulting p_{know} is stored in local storage and dispatched asynchronously to Neo4j via `PUT /quiz/mastery`.

3. Downstream Personalization Delivery:

- *Weighted Quiz Selection:* Multi-topic quizzes sample questions with weights inversely proportional to topic mastery ($\propto 1 - p_{\text{know}}$).
- *Flashcard Weak-Topic Infill:* Practice decks prioritize review cards from topics exhibiting the lowest mastery scores.
- *Adaptive Nudges & Progress:* The backend `GET /students/me/nudge` endpoint inspects `:MASTERS` edges to surface high-priority study recommendations on the dashboard, while `SubjectsView` renders real-time topic progress bars.

Flow 6: Offline Multimodal Ingestion & MCQ Generation Pipeline

The offline ingestion pipeline converts unstructured medical textbooks into validated, high-yield multiple-choice questions across five sequential stages:

1. **Stage 1: Parsing & Normalization:** The MinerU CLI (isolated in a Python 3.10 virtual environment, `.venv-mineru`) processes textbook PDFs to extract layout bounding boxes (`_middle.json`) and crop embedded anatomical figures and diagrams (`images/*.jpg`).

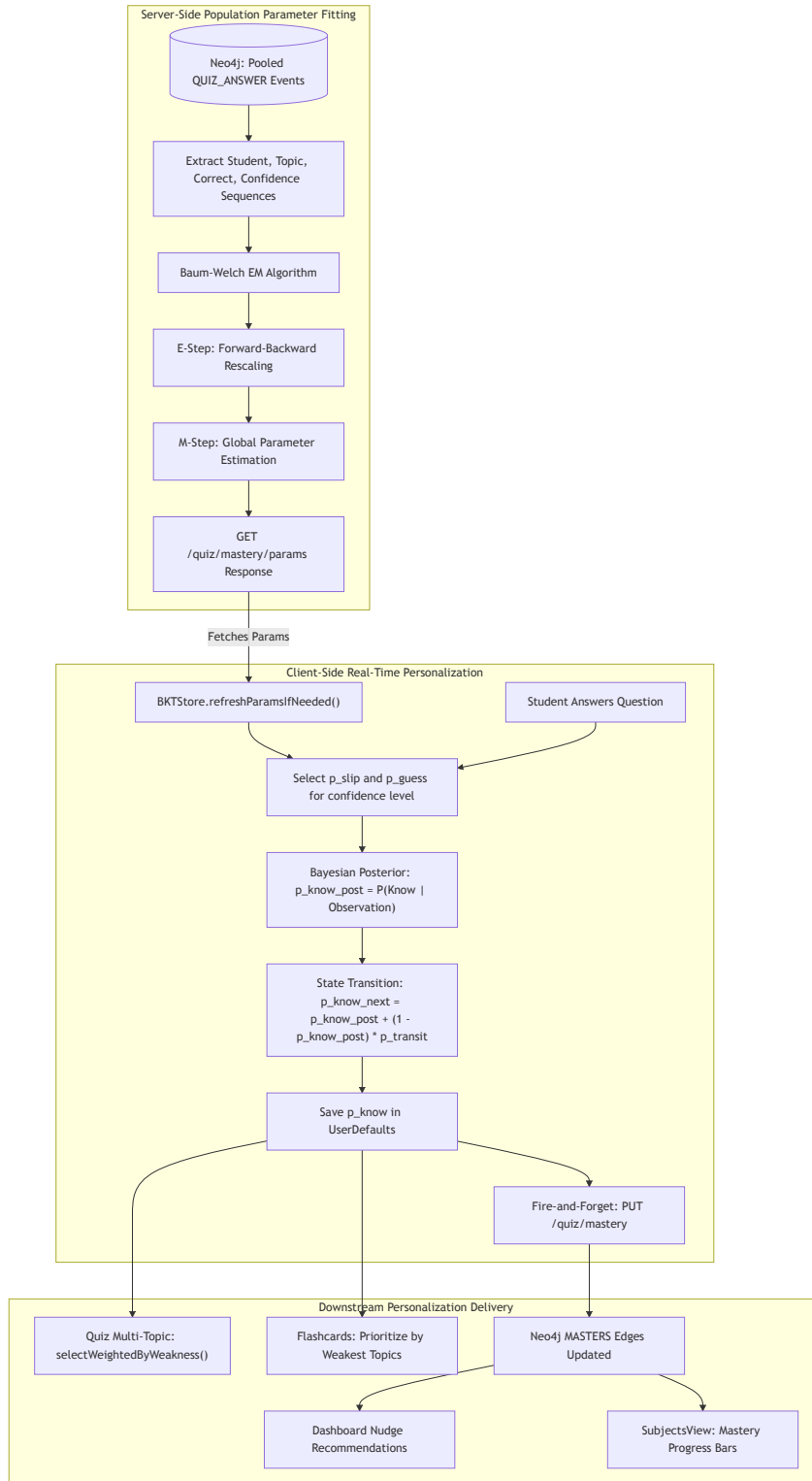


Figure 5: Hybrid BKT personalization loop: server-side Baum-Welch EM parameter fitting and client-side real-time knowledge tracing on iPadOS.

2. **Stage 2: 8-Stage Post-Processing Engine** (`src/post_process`): Performs deterministic schema validation, layout block flattening, typographic/geometrical feature extraction, header/footer repetition filtering, rule-based block classification (Title, Section, Paragraph, Table), cross-page reading order reconstruction, and semantic tree construction (`_post_processed.json`).
3. **Stage 3: Multimodal Captioning & Semantic Chunking**: `Qwen2.5-VL-7B-Instruct` generates detailed descriptions and clinical transcriptions for visual crops. `unify.py` merges textual AST and visual captions into unified Markdown, which is segmented into context-enriched chunks (`_v1_chunks.json`) via `all-MiniLM-L6-v2` semantic boundary embeddings.
4. **Stage 4: 6-Step LLM MCQ Generation** (`notebooks/mcq_generation.ipynb`):
 - *Step 1 (Chunk Filtering)*: Prepends hierarchical context headings to each chunk.
 - *Step 2 (Triple Extraction)*: Uses `Qwen3.5-27B` / `HuatuoGPT-o1-72B` to extract biomedical triples over 13 core clinical relations.
 - *Step 3 (Distractor Synthesis)*: Employs a two-tier strategy retrieving in-corpus medical entities with LLM fallback generation.
 - *Step 4 (Stem Generation)*: Constructs question stems across Recall, Application, and Clinical Vignette modalities.
 - *Step 5 (Self-Critique)*: Executes a 1-cycle verification prompt checking factual accuracy and key uniqueness, applying minimal-edit repairs.
 - *Step 6 (Option Shuffling & UID Generation)*: Shuffles options and serializes to `question_bank.json`.
5. **Stage 5: Database Population**: `scripts/populate_mcq_postgres.py` ingests validated questions, distractors, and explanations directly into the PostgreSQL `mcq_questions` table.

Flow 7: Domain Knowledge Graph (medkg) Pipeline

The domain knowledge graph pipeline constructs a formal, ontology-grounded biomedical semantic graph across eight stages:

1. **Stage 1 (Parse)**: Ingests semantically tagged Markdown containing clinical inline annotations (`<con>`, `<def>`, `<clin>`), validating and normalizing character spans.
2. **Stage 2 (Extract)**: Performs Named Entity Recognition via `scispaCy` (`en_core_sci_scibert`) and biomedical relation extraction using `HuatuoGPT-o1-72B` over a 27-relation medical ontology.
3. **Stage 3 (Link)**: Maps extracted entity surface forms to standard UMLS/SNOMED CT Concept Unique Identifiers (CUIs) using `SapBERT` dense embeddings indexed in FAISS (`IndexFlatIP`, cosine similarity threshold ≥ 0.70).
4. **Stage 4 (Post-coordination)**: Mints scoped `:INSTANCE` nodes to model complex clinical modifiers and qualifiers without combinatorial ontology explosion.
5. **Stage 5 (Multimodal Grounding)**: Connects visual figure captions and anatomical depictions to corresponding concept nodes.

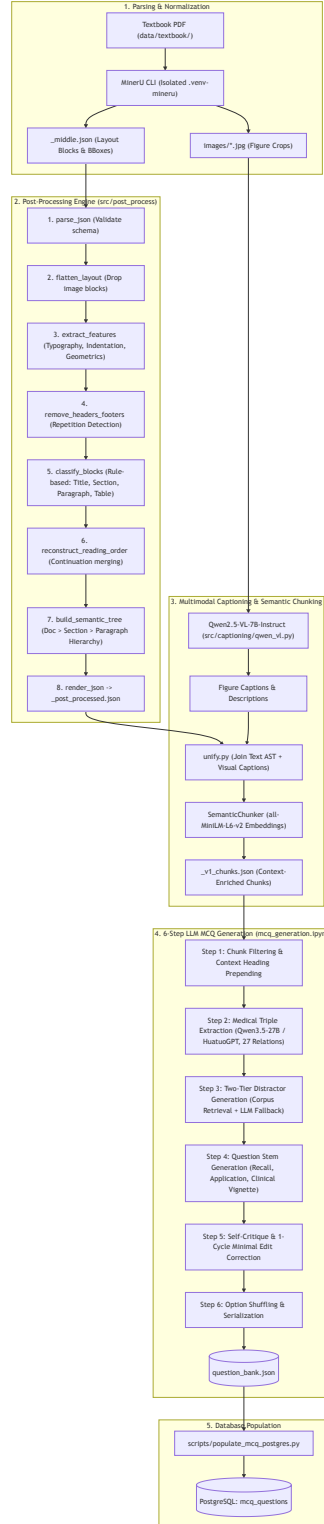


Figure 6: 5-stage offline ingestion pipeline: PDF parsing, 8-stage post-processing, multimodal captioning, 6-step LLM MCQ generation, and PostgreSQL population.

6. **Stage 6 (RDF Assertion):** Serializes extracted assertions into W3C RDF Named Quads (`<subject> <predicate> <object> <graph>`) with provenance metadata.
7. **Stage 7 (OWL-RL Reasoning):** Applies deterministic OWL 2 RL inference rules to materialize implicit taxonomic, inverse, and transitive relationships.
8. **Stage 8 (Serving):** Exports the reasoned knowledge base as N-Quads, hosts a SPARQL query endpoint, and synchronises triples into Neo4j via Neosemantics (`n10s`) to empower graph-grounded MCQ generation and semantic exploration.



Figure 7: 8-stage medkg pipeline: Parse $\rightarrow$ NER $\rightarrow$ Entity Linking $\rightarrow$ Post-coordination $\rightarrow$ RDF Assertion $\rightarrow$ OWL-RL Reasoning $\rightarrow$ Serve (N-Quads / SPARQL / Neo4j n10s).

Model Deployment Method

Model Zoo & Task Allocation Matrix

Model	Type	Task & Responsibility	Runtime / Env
HuatuoGPT-o1-72B (4-bit)	Medical LLM (Reasoning)	Document tagging & 27-relation biomedical triple extraction	vLLM OpenAI-compatible endpoint
Qwen3.5-27B	General / Medical LLM	Triple extraction, distractor synthesis, stem generation, self-critique	vLLM / Local HuggingFace
Qwen2.5-VL-7B-Instruct	Vision-Language Model	Medical figure captioning & diagram transcription	PyTorch / Accelerate (MPS / CUDA)
SapBERT (cambridgeltl)	Biomedical Sentence Transformer	Entity linking to UMLS/SNOMED CT concepts via FAISS ($\cos \geq 0.70$)	FAISS IndexFlatIP (CPU)
all-MiniLM-L6-v2	Sentence Transformer	Semantic boundary chunking of unified textbook text	sentence-transformers (CPU / MPS)
MedGemma 4B + UNet	Multimodal VLM + CNN	Diagram classifier & leader line endpoint detector	mlx-community / PyTorch
Baum-Welch EM Engine	2-State HMM	Global parameter fitting for Knowledge Tracing	NumPy / SciPy (Pure CPU)
On-Device BKT Engine	2-State HMM Forward Updater	Real-time student mastery calculation (p_{know}) on iPadOS	Swift Standard Library (Zero Network)

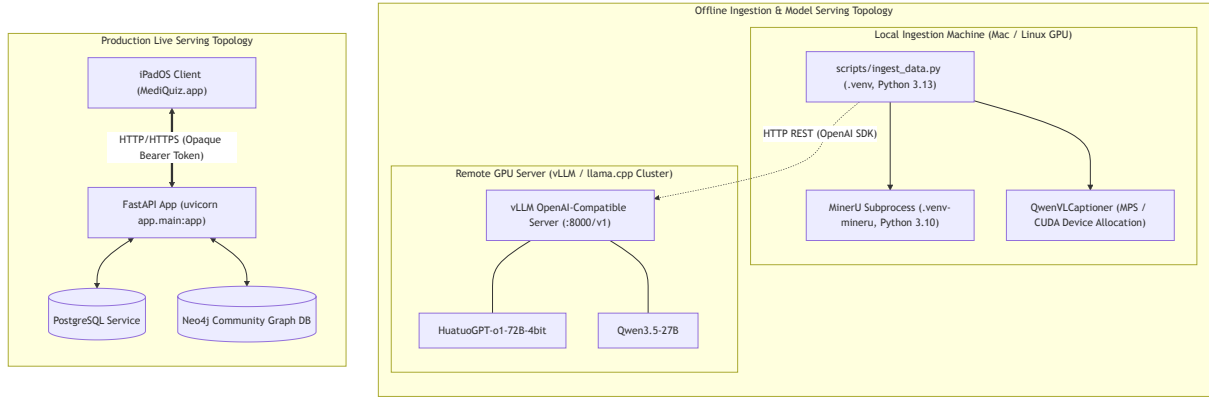


Figure 8: Deployment topology: Offline Ingestion infrastructure (local Mac / remote GPU cluster) and Production Live Serving infrastructure (FastAPI + PostgreSQL + Neo4j).

Deployment Topologies & Inference Infrastructure

Zero-Inference Live Serving Architecture

A critical architectural achievement is that **the live FastAPI serving application performs zero model inference at runtime**:

1. **CPU-Only Bound:** The production server requires only standard CPU instances (e.g. 2 vCPUs, 4 GB RAM) with negligible footprint.
2. **Instant Latency:** Response times for `/quiz/questions`, `/quiz/check`, and `/quiz/log` remain sub-15 ms because they execute indexed PostgreSQL queries and atomic Cypher graph transactions without model queue delays.
3. **High Reliability:** Server stability is completely decoupled from GPU drivers, CUDA version mismatches, or VRAM exhaustion.

Multi-Environment Isolation Strategy

Because several specialised medical AI libraries carry incompatible Python and C-extension dependencies, the project isolates runtimes into three dedicated virtual environments:

- **.venv/** — Main Environment: Python ≥ 3.13 (managed by `uv`). Dependencies: FastAPI, Uvicorn, Neo4j, SQLAlchemy, PyTorch, Transformers, Accelerate, Pydantic, SlowAPI.
- **.venv-mineru/** — Isolated Environment: Python 3.10/3.11. Dependencies: `mineru`, `magic-pdf`, `Detron2`, `PaddlePaddle`. Invoked exclusively as a subprocess from `scripts/ingest_data.py`.
- **Domain KG env** — Isolated Environment: Python 3.9–3.11. Dependencies: `scispaCy`, `spacy < 3.8`, `faiss-cpu`, `rdfliib`, `owlrl`. Required for Stage 2 NER and Stage 3 UMLS linking.

Hardware Optimisation & Native Runtime Guards

- **Dynamic Device Selection with MPS Bug Workarounds** (`src/captioning/qwen_v1.py`): Dynamically allocates execution targets (CUDA $\rightarrow$ MPS)

→ CPU) and implements explicit memory purging (`torch.mps.empty_cache()`, `gc.collect()`) upon completion to operate smoothly within Apple Silicon Unified Memory constraints.

- **Native macOS Segfault Protection** (`src/domain_kg/native_env.py`): Addresses an issue on macOS where PyTorch, Thinc/Blis (spaCy), and FAISS clash when initialising separate OpenMP thread pools, resulting in Segmentation fault: 11. Setting `MEDKG_NATIVE_THREADS=1` and `KMP_DUPLICATE_LIB_OK=TRUE` at the top of the package before any C-extensions load prevents segmentation crashes.

Source Code Structure

Backend Codebase Organization (fastapi-backend)

```
intelligent-tutoring-system-for-medical-student/
├── app/
│   ├── main.py                # FastAPI HTTP Layer & Routers
│   ├── auth_middleware.py     # Entry point, middleware, CORS, routers
│   ├── dependencies.py        # SessionAuthMiddleware: Allowlist & token resolver
│   ├── errors.py              # Neo4j driver injection & get_current_student_id
│   ├── limiter.py             # Unified error envelope & global exception handlers
│   ├── openapi.py             # SlowAPI rate limiter configuration
│   ├── schemas.py             # OpenAPI schema generator & error response docs
│   └── routers/
│       ├── students.py        # Pydantic DTO request/response contracts
│       ├── quiz.py            # Auth, profile, streak, energy, nudge endpoints
│       └── flashcards.py       # Topics, questions, check/log, sessions, mastery
│                               # Cards, reviews, sessions, due-review queue
├── src/
│   ├── student_kg/            # Core Business Logic & Domain Services
│   │   ├── driver.py          # Neo4j Student Knowledge Graph Drivers
│   │   ├── enrollment.py      # Neo4j singleton driver & constraint initialization
│   │   ├── session.py         # Student creation & ENROLLMENT genesis event
│   │   ├── streak.py          # Auth session creation, validation & revocation
│   │   └── energy.py          # UTC daily streak calculation & energy restore
│   │                               # Gamification currency ledger
│   ├── quiz/                 # Quiz Subsystem
│   │   ├── bank.py            # PostgreSQL question bank loader & sanitizer
│   │   ├── attempts.py        # Event-sourced QUIZ_ANSWER & spaced repetition
│   │   ├── sessions.py        # QuizSession start, end, cancel & history
│   │   ├── mastery.py         # Student-Topic :MASTERS persistence layer
│   │   └── bkt_fit.py          # Baum-Welch EM Knowledge Tracing fitter
│   ├── flashcards/           # Flashcard Subsystem
│   │   ├── cards.py           # Bank-derived flashcard projection
│   │   ├── reviews.py         # 4-tier review logger & SuperMemo scheduler
│   │   └── sessions.py        # FlashcardSession lifecycle manager
│   ├── post_process/          # 8-Stage Deterministic MinerU Cleanup Pipeline
│   │   ├── pipeline.py        # 8-stage coordinator
│   │   └── [stages 1-8 modules]
│   ├── captioning/
│   │   └── qwen_vl.py          # Qwen2.5-VL-7B vision-language captioner
│   ├── ingestion/
│   │   ├── unify.py           # Merge text AST + visual captions
│   │   └── semantic_chunk.py   # Semantic boundary chunker (MiniLM)
│   └── domain_kg/             # 8-Stage Biomedical KG Pipeline (medkg)
│       ├── native_env.py      # macOS OpenMP segfault workaround
│       └── ontology.py         # 27-relation biomedical ontology loader
```

- └─ guards.py # 14 deterministic relation consistency guards
- └─ stages/ # Stages 1-8 (Parse -> NER -> Link -> Serve)
- └─ scripts/ # CLI Ingestion & Seeding Scripts
 - └─ ingest_data.py # Production CLI: PDF -> Chunks pipeline runner
 - └─ populate_mcq_postgres.py # Loads question JSON into PostgreSQL
 - └─ generate_dummy_interactions.py # Synthetic student interaction seeder
- └─ notebooks/ # Exploratory & Generation Notebooks
 - └─ mcq_generation.ipynb # 6-step LLM question generation & self-critique
 - └─ knowledge_tracing.ipynb # BKT validation & Baum-Welch EM research
 - └─ mcq_output/question_bank.json # Canonical question bank (188 MCQs)
- └─ docker-compose.yml # Local infrastructure (Postgres, Neo4j, MinIO)
- └─ pyproject.toml & uv.lock # UV project definition & pinned lockfile

Frontend Codebase Organization (ipados-swiftui)

```
personalized-learning-ios/personalized-medical-learning-ios/
├─ App/ # App Lifecycle & Root Navigation
│   ├── ...App.swift # @main entry point
│   ├── ContentView.swift # Auth routing: RootView or LoginView
│   └── RootView.swift # Sidebar navigation container & active view switch
├─ Core/ # Shared Infrastructure & Foundations
│   ├── Networking/
│   │   ├── APIConfig.swift # Base URL configuration
│   │   ├── APIClientProtocol.swift # Complete async protocol abstraction
│   │   └── APIClient.swift # URLSession singleton + Bearer auth
│   ├── Persistence/
│   │   ├── KeychainStore.swift # Security framework wrapper
│   │   ├── SessionManager.swift # Session token & expiry facade
│   │   └── BKTStore.swift # On-device Bayesian Knowledge Tracing engine
│   └── UI/
│       ├── SidebarView.swift # iPad landscape sidebar navigation
│       └── TopBarView.swift # Header with energy & student indicators
└─ Features/ # Domain Feature Modules (MVVM)
    ├── Auth/ # LoginView, RegisterView, ViewModels
    ├── Dashboard/ # DashboardView, Streak/Nudge ViewModels
    ├── Quiz/ # QuizView, QuizSetupView, QuizViewModel, ...
    ├── Flashcard/ # FlashcardView, FlashcardSetupView, ...
    ├── Mastery/ # MasteryAPI, BKT parameter sync
    ├── Subjects/ # SubjectsView, SubjectDetailView
    ├── History/ # HistoryView, HistoryViewModel
    ├── Bookmark/ # BookmarkView
    └── Settings/ # SettingsView (backend endpoint, logout)
```

Cross-Repository API & Protocol Contract Mapping

HTTP Route	Method	Auth	Primary Database Action
/students/register	POST	None	Creates Student & genesis ENROLLMENT in Neo4j
/students/login	POST	None	Creates :Session node in Neo4j
/students/logout	POST	Bearer	Sets Session.revoked_at in Neo4j

HTTP Route	Method	Auth	Primary Database Action
/students/me/profile	GET	Bearer	Reads Student node from Neo4j
/students/me/streak	GET	Bearer	Evaluates UTC activity dates from Neo4j
/students/me/streak/restore	POST	Bearer	Spends 10 Energy to bridge 1 missed day
/students/me/energy	GET	Bearer	Reads Student.energy from Neo4j
/students/me/nudge	GET	Bearer	Finds soonest due quiz & flashcard items
/quiz/topics	GET	None	Reads distinct topic_tag from PostgreSQL
/quiz/topics/{path}/questions	GET	None	Reads sanitised questions from PostgreSQL
/quiz/questions/{uid}/check	POST	None	In-memory answer verification (Stateless)
/quiz/topics/{path}/sessions	POST	Bearer	Creates QuizSession node in Neo4j
/quiz/questions/{uid}/log	POST	Bearer	Creates QUIZ_ANSWER & updates REVIEWING edge
/quiz/sessions/{id}/end	POST	Bearer	Aggregates score, awards Energy in Neo4j
/quiz/review/due	GET	Bearer	Queries REVIEWING edges where next_review_at $\leq$ now
/quiz/mastery	PUT	Bearer	Overwrites :MASTERS edge p_know in Neo4j
/quiz/mastery/params	GET	Bearer	Runs / caches Baum-Welch EM fit on pooled events
/flashcards/cards	GET	None	Projects questions to flashcard models
/flashcards/cards/{uid}/reveal	POST	None	Returns card back text & explanation
/flashcards/sessions	POST	Bearer	Creates FlashcardSession in Neo4j
/flashcards/cards/{uid}/log	POST	Bearer	Creates FLASHCARD_REVIEW & updates Flashcard node
/flashcards/sessions/{id}/end	POST	Bearer	Marks FlashcardSession.status = 'completed'
/flashcards/review/due	GET	Bearer	Queries Flashcard nodes where next_review_at $\leq$ now

Mathematical Foundations & Algorithms

Bayesian Knowledge Tracing (BKT) with Confidence Emissions

Standard BKT models a student's latent knowledge of a specific medical topic as a 2-state Hidden Markov Model:

- **State 0:** *Doesn't Know*
- **State 1:** *Knows*

To account for metacognitive awareness, emission probabilities p_{slip} and p_{guess} are conditioned on the student's self-reported confidence $c \in \{\text{confident, unsure, guessing}\}$:

Confidence (c)	p_{slip}	p_{guess}	Pedagogical Interpretation
Confident	0.05	0.10	Confident-wrong strongly penalises p_{know} as a misconception.
Unsure	0.10	0.25	Standard baseline BKT behaviour.
Guessing	0.30	0.50	Correct guess does not falsely award mastery.

Observation Update (Bayes Rule): Given prior mastery p_{know} and observation $O_t \in \{\text{Correct}, \text{Incorrect}\}$ with confidence c :

$$O_t = \text{Correct} : \quad p_{\text{know}}^{\text{post}} = \frac{p_{\text{know}} \cdot (1 - p_{\text{slip}}[c])}{p_{\text{know}} \cdot (1 - p_{\text{slip}}[c]) + (1 - p_{\text{know}}) \cdot p_{\text{guess}}[c]}$$

$$O_t = \text{Incorrect} : \quad p_{\text{know}}^{\text{post}} = \frac{p_{\text{know}} \cdot p_{\text{slip}}[c]}{p_{\text{know}} \cdot p_{\text{slip}}[c] + (1 - p_{\text{know}}) \cdot (1 - p_{\text{guess}}[c])}$$

Learning State Transition (accounting for knowledge acquisition during the attempt):

$$p_{\text{know}}^{\text{next}} = p_{\text{know}}^{\text{post}} + (1 - p_{\text{know}}^{\text{post}}) \cdot p_{\text{transit}}$$

Baum-Welch Expectation-Maximisation (EM) Parameter Fitting

Implemented in `src/quiz/bkt_fit.py`, the Baum-Welch EM algorithm fits optimal global parameters across all pooled student sequences:

1. **E-Step (Forward-Backward Rescaling):** Compute smoothed posterior state probabilities $\gamma_t(i) = P(S_t = i \mid \mathbf{O})$ and transition probabilities $\xi_t(0, 1) = P(S_t = 0, S_{t+1} = 1 \mid \mathbf{O})$.
2. **M-Step (Closed-Form Aggregation):**

$$p_{\text{init}} = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \gamma_0^{(s)}(1) \quad p_{\text{transit}} = \frac{\sum_s \sum_t \xi_t^{(s)}(0, 1)}{\sum_s \sum_t \gamma_t^{(s)}(0)}$$

$$\forall c : p_{\text{slip}}[c] = 1 - \frac{\sum_s \sum_{t: O_t=\text{Correct}, C_t=c} \gamma_t^{(s)}(1)}{\sum_s \sum_{t: C_t=c} \gamma_t^{(s)}(1)} \quad p_{\text{guess}}[c] = \frac{\sum_s \sum_{t: O_t=\text{Correct}, C_t=c} \gamma_t^{(s)}(0)}{\sum_s \sum_{t: C_t=c} \gamma_t^{(s)}(0)}$$

Dual-Track Spaced Repetition Scheduling

The two tracks use distinct scheduling mechanics:

- **Quiz Track** ((Student)-[:REVIEWING]->(Question) edge): Correct + Confident $\Rightarrow$ streak += 1, interval = min(60, interval×2). Wrong or low-confidence $\Rightarrow$ reset: streak = 0, interval = 1.
- **Flashcard Track** (Flashcard node): *Again* $\Rightarrow$ reset to 1 day; *Hard* $\Rightarrow$ interval $\times$ 1.2; *Good* $\Rightarrow$ streak +1, interval $\times$ 2.0; *Easy* $\Rightarrow$ streak +1, interval $\times$ 2.5. All capped at 60 days.

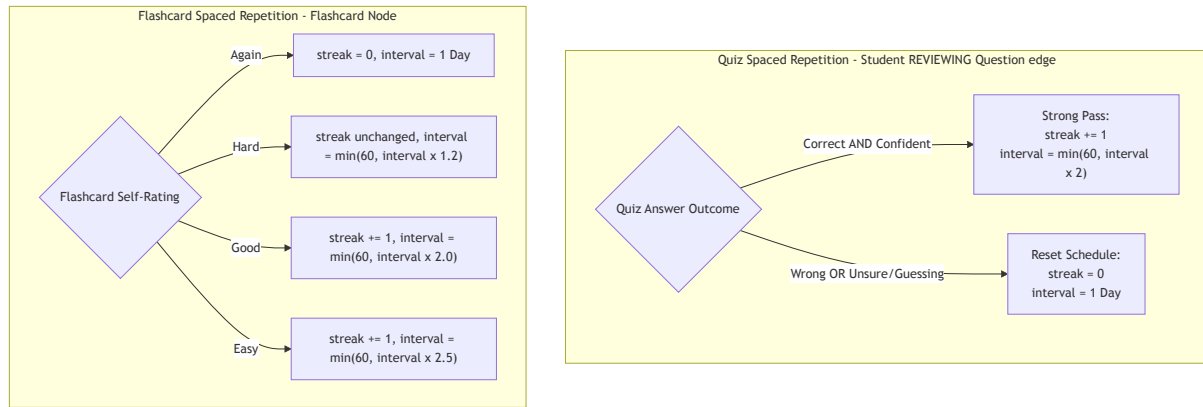


Figure 9: Dual-track spaced repetition: Quiz track (REVIEWING edge) and Flashcard track (SuperMemo self-rating intervals).

Gamification Economy: Streaks, Energy & Nudge Heuristics

1. **Consecutive Streak Tracking** (`src/student_kg/streak.py`): Evaluates distinct calendar days in UTC where the student generated at least one `QUIZ_ANSWER` or `FLASHCARD_REVIEW`. A gap ≥ 2 days resets `current_streak = 0` while `previous_streak` retains the broken length.
2. **Streak Restoration** (`POST /students/me/streak/restore`): If the student missed exactly 1 day, they spend **10 Energy** units to bridge the gap and restore their previous streak.
3. **Energy Ledger** (`src/student_kg/energy.py`): Earned: +5 per Quiz Session completion, +1 per Flashcard Review. Spent: -10 per Streak Restore.
4. **Nudge Recommendation Engine** (`GET /students/me/nudge`): Aggregates total due quiz questions and due flashcards, identifying the single item with `min(next_review_at)` across both modes to populate the top dashboard reminder card.

Operations, Runbook & Local Setup

Backend & Infrastructure Bootstrapping

Prerequisites: macOS or Linux, uv package manager, Docker & Docker Compose.

1. Setup Environment & Databases:

```
# Navigate to backend repo
cd intelligent-tutoring-system-for-medical-student

# Configure environment variables
cp .env.example .env

# Start Neo4j, PostgreSQL, and MinIO containers
docker compose up -d postgres neo4j minio
```

```
# Synchronize Python dependencies via uv
uv sync
```

2. Populate Seed Data:

```
# Populate PostgreSQL mcq_questions table
uv run python scripts/populate_mcq_postgres.py

# (Optional) Populate synthetic student interactions
uv run python scripts/generate_dummy_interactions.py
```

3. Launch Live FastAPI Server:

```
uv run uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Verify: `curl http://localhost:8000/health` → `{"status":"ok"}`. Interactive Swagger docs: `http://localhost:8000/docs`.

iPadOS Client Setup & Compilation

Prerequisites: Xcode 16.0+ (iPadOS 26.5+ SDK); Simulator or Physical iPad with Developer Mode enabled.

1. Configure Host IP in Core/Networking/APIConfig.swift:

```
enum APIConfig {
    // Set to local development machine IP on LAN
    static let baseURL = URL(string: "http://10.67.52.231:8000")!
}
```

2. Open Project & Run:

```
cd personalized-learning-ios
open personalized-medical-learning-ios.xcodeproj
```

Select an iPad simulator (e.g. iPad Pro 13-inch) in **Landscape Orientation** and press `Cmd + R`.

Dataset Ingestion & Content Regeneration

To process new textbook PDFs through the multimodal pipeline:



Figure 10: Dataset ingestion runbook: PDF → ingest script → MCQ notebook → PostgreSQL population.

```
# 1. Run MinerU + Post-Process + Captioning + Chunking
uv run python scripts/ingest_data.py --pdf data/textbook/Ganongs_Physiology.pdf

# 2. Open and run notebooks/mcq_generation.ipynb end-to-end
# Generates notebooks/mcq_output/question_bank.json

# 3. Reload PostgreSQL question table
uv run python scripts/populate_mcq_postgres.py
```

Verification & Automated Test Suite

```
# Run backend pytest suite
cd intelligent-tutoring-system-for-medical-student
uv run pytest -v
```

Developer Extension Guide & Future Roadmap

Recipe: Adding a New Backend Endpoint

To expose a new API endpoint (e.g. `POST /students/me/bookmarks`):

1. **Define Schema:** Add request/response Pydantic models to `app/schemas.py`.
2. **Implement Domain Logic:** Add graph query functions in `src/student_kg/`.
3. **Register Router Route:** Add a route handler to `app/routers/students.py` specifying `@router.post(...)`.
4. **Configure Auth Allowlist (If Public):** If unauthenticated, update regex rules in `app/auth_middleware.py`.
5. **Add Tests:** Create corresponding test cases in `tests/`.

Recipe: Adding a New iPadOS Feature / View

To implement a new feature view (e.g. `BookmarksView`):

1. **Declare Protocol Method:** Add the async API signature to `Core/Networking/APIClientProtocol.swift`.
2. **Implement Network Call:** Implement the method in `Features/Bookmark/BookmarkAPI.swift` using `client.get(...)` or `client.send(...)`.
3. **Build ViewModel:** Create `BookmarkViewModel` conforming to `ObservableObject` marked `@MainActor`, holding `@Published` state and injecting any `APIClientProtocol`.
4. **Construct SwiftUI View:** Build the interface in `BookmarkView.swift` observing `BookmarkViewModel`.
5. **Register Navigation:** Add an enum case to `SidebarItem` in `Core/UI/SidebarView.swift` and wire the switch statement in `App/RootView.swift`.

Limitations and Future Improvements

Limitations

- **Absence of Bahasa Indonesia Localization (Monolingual English Pipeline):** The offline knowledge extraction and question generation pipelines (utilising HuatuoGPT-o1-72B and Qwen3.5-27B) as well as the client user interface operate exclusively in English. Consequently, the platform cannot directly ingest Indonesian-language medical textbooks or support students who prefer native-language instructional materials and localized clinical terminology.
- **Absence of Multimodal and Visual Question Generation:** Medical education relies heavily on visual comprehension, including anatomical diagrams, histological micrographs, diagnostic imaging, and biochemical pathways. While the ingestion pipeline parses structural headings and textual narratives from source textbooks, non-textual figures are currently stripped during document cleaning. The system is therefore unable to formulate visual questions or multimodal clinical case questions that require interpreting images, charts, and diagrams.
- **Cold-Start Latency in Bayesian Knowledge Tracing (BKT):** The student personalization engine leverages Bayesian Knowledge Tracing to infer latent mastery probabilities across granular medical concepts. Because the model relies on historical practice trajectories, newly enrolled students lack sufficient interaction records. As a result, initial mastery estimations default to uncalibrated global priors ($P(L_0)$), leading to suboptimal question recommendations and difficulty matching until a critical mass of formative responses is gathered.

Future Improvements

- **Multilingual Support & Indonesian Medical NLP:** Extend the ingestion, knowledge extraction, and MCQ generation pipelines to support Bahasa Indonesia. This includes integrating localized medical terminologies and ontology mappings (such as SNOMED CT Indonesian translations), fine-tuning bilingual extraction prompts, and providing full localization across the iPadOS application interface.
- **Multimodal & Image-Grounded Question Synthesis:** Incorporate vision-language models (VLMs) alongside document layout parsing to extract, index, and associate high-yield anatomical illustrations and histological slides. This will enable automated generation of visual diagnostic questions, image-label matching tasks, and hot-spot identification flashcards.
- **Expanded Question Typologies & Granular Difficulty Calibration:** Introduce a wider variety of question structures, including multi-step clinical reasoning vignettes, case-based scenarios, and assertion-reason questions, while retaining the five-option multiple-choice format. Additionally, incorporate cognitive taxonomy classifiers, such as Bloom’s Revised Taxonomy, to enable more precise difficulty calibration and stronger alignment with medical board examination standards.
- **Multi-Signal & Deep Knowledge Tracing Enhancements:** Enhance the mastery inference framework by transitioning from standard single-parameter BKT toward multi-signal or Deep Knowledge Tracing (DKT) architectures. Incorporating continuous behavioral

signals—such as response latency, self-assessed confidence scores, circadian study intervals, and knowledge graph prerequisite traversal—will yield significantly more accurate student models.

- **Expanded Multi-Format Document Ingestion:** Broaden the preprocessing and OCR ingestion layer beyond textbook PDFs to handle additional institutional formats, including Microsoft Word (.docx) lecture notes, slide presentations (.pptx), clinical practice guidelines, web articles, and scanned paper-based exam archives with formula and tabular data recovery.